

MentalHealthOS

O **MentalHealthOS** (HealthOS) é um ecossistema abrangente e modular focado no domínio da saúde mental, integrando modelos de linguagem (LLMs), fluxos de processamento clínico (pipelines) e interfaces nativas (App macOS e CLI) em uma única plataforma.

Ele foi projetado para oferecer a profissionais e pacientes um ambiente seguro e orquestrado por agentes, lidando com transcrições, processamento de linguagem natural, revisões clínicas e arquitetura de dados longitudinais.

Ecossistema (Mindmap)

```
mindmap
  root((HealthOS))
    Interfaces
      HealthOSApp (macOS SwiftUI)
      HealthOSCLI (Terminal)
    Motor Clínico
      PipelineEngine
      Transcription
      LLM Stage Runners
    Agentes (LLM)
      ConversationOrchestrator
      SafetyReviewAgent
      PatientCareAgent
      ContextArchitect
    Modelos & Dados
      Session
      ConsultationIntake
      Workspaces
```

Arquitetura do Sistema

O MentalHealthOS adota uma arquitetura híbrida e modular, compondo pacotes Swift nativos para a camada de visualização, execução e orquestração base, enquanto também integra rotinas especializadas (como agentes e processamento) através de runtimes em TypeScript/Node.js e integração com Codex/LLMs.

```
graph TD
  subgraph Interfaces [Interfaces de Usuário]
    A[HealthOSApp <br/> SwiftUI macOS]
    B[HealthOSCLI <br/> Command Line]
  end

  subgraph CoreSwift [Ecossistema Swift]
    C[HealthOSUI <br/> Componentes de UI]
    D[HealthOSTerminal <br/> Bridge de Terminal e CLI]
    E[HealthOSPipeline <br/> Processamento e LLM Runtime]
    F[HealthOSAgents <br/> Orquestração e Registro]
    G[HealthOSCore <br/> Modelos e Workspaces]
  end

  subgraph Runtimes [Runtimes e Agentes]
    H[TS/Node Runtimes <br/> chat-cli]
    I[Modelos LLM <br/> Claude, Codex, etc]
    J[Catálogo de Agentes <br/> src/agents/]
  end
```

```
A --> C
A --> E
A --> F
A --> G

B --> D
B --> E
B --> F
B --> G

E --> H
F --> H
H --> I
H --> J
F --> J
```

Estrutura de Pacotes Swift (SwiftPM)

A separação de responsabilidades no backend Swift é feita através de pacotes modulares que compõem o motor principal do HealthOS.

```
classDiagram
class HealthOSApp {
+ContentView
+AppState
}
class HealthOSCLI {
+main()
}
class HealthOSCore {
+Models: Session, ConsultationIntake
+WorkspaceManager
+Config
}
class HealthOSPipeline {
+PipelineEngine
+LLMStageRunners
+TranscriptionProvider
+ClaudeCodeRunner
}
class HealthOSAgents {
+ConversationOrchestrator
+AgentRegistry
+ChatContext
+WorkflowDetection
}
class HealthOSUI {
+AgentListView
+ConversationView
+MessageBubble
}
class HealthOSTerminal {
+TerminalBridge
+PipelineStageRunner
}

HealthOSApp --> HealthOSUI
HealthOSApp --> HealthOSPipeline
HealthOSApp --> HealthOSAgents
HealthOSCLI --> HealthOSTerminal
HealthOSCLI --> HealthOSPipeline
HealthOSUI --> HealthOSCore
HealthOSPipeline --> HealthOSCore
```

```
Health0SAgents --> Health0SCore
Health0STerminal --> Health0SCore
```

Workflow dos Agentes LLM

O `ConversationOrchestrator` é o coração da comunicação. Ele recebe as intenções e roteia para o catálogo de agentes (`src/agents/`), gerenciando a sessão e garantindo que análises clínicas e protocolos de segurança sejam validados antes da resposta final.

```
sequenceDiagram
    participant User as Usuário / Paciente
    participant Orchestrator as ConversationOrchestrator
    participant Context as ContextAgent
    participant Clinical as PatientCare / SessionPrep
    participant Safety as SafetyReviewAgent
    participant LLM as LLM Runtime / chat-cli

    User->>Orchestrator: Envia mensagem/tarefa
    activate Orchestrator

    Orchestrator->>Context: Solicita contexto (Histórico, Artefatos)
    Context-->>Orchestrator: Retorna `AgentContext`

    Orchestrator->>Clinical: Roteia intenção (AgentTask)
    activate Clinical
    Clinical->>LLM: Injeta definições e processa prompt
    LLM-->>Clinical: Retorna rascunho de resposta/análise
    Clinical-->>Orchestrator: Retorna `AgentResult`
    deactivate Clinical

    Orchestrator->>Safety: Solicita revisão de risco/segurança
    activate Safety
    Safety->>LLM: Valida compliance
    LLM-->>Safety: Aprova/Ajusta
    Safety-->>Orchestrator: Resposta Segura Validada
    deactivate Safety

    Orchestrator-->>User: Retorna resposta sintetizada e grava na memória
    deactivate Orchestrator
```

Processamento do Pipeline

O módulo `Health0SPipeline` controla a ingestão e transformação de dados, como transcrições de áudio e análise de prontuários.

```
stateDiagram-v2
    [*] --> Intake

    state Intake {
        AudioUpload --> Transcription
        TextEntry --> Parsing
    }

    Intake --> Processing

    state Processing {
        Transcription --> DeterministicStage
```

```

    Parsing --> DeterministicStage
    DeterministicStage --> LLMStageRunners
}

Processing --> Review

state Review {
    LLMStageRunners --> ClinicalReview
    ClinicalReview --> SafetyCheck
}

Review --> [*] : Artefato Gerado (PipelineJSON)

```

17 Timeline de Execução do Pipeline (Gantt)

Abaixo, ilustramos o paralelismo de como um estágio de processamento pesado da Pipeline de entrada acontece.

```

gantt
    title Execução de Pipeline Clínica
    dateFormat YYYY-MM-DD
    axisFormat %H:%M
    section Ingestão
    Upload de Áudio :a1, 2026-06-08, 2m
    Transcrição (Whisper) :a2, after a1, 5m
    section Modelagem
    Extração Determinística :b1, after a2, 1m
    LLM Contextualization :b2, after b1, 3m
    section Validação
    Clinical Safety Review :c1, after b2, 2m
    Geração de Artefato :c2, after c1, 1m

```

Modelagem de Dados Core

Modelos principais mantidos pelo Health05Core em Swift, usados em todos os Workspaces.

```

erDiagram
    PATIENT ||--o{ SESSION : participates
    PROFESSIONAL ||--o{ SESSION : conducts
    SESSION ||--|| CONSULTATION_INTAKE : contains
    SESSION ||--o{ AGENT_EVENT : triggers

    PATIENT {
        UUID id
        String name
        Date birthDate
    }

    PROFESSIONAL {
        UUID id
        String name
        String specialty
    }

    SESSION {
        UUID id
        Date startTime
        String status
    }

```

```
CONSULTATION_INTAKE {  
    UUID id  
    String chiefComplaint  
    String clinicalNotes  
}  
  
AGENT_EVENT {  
    UUID id  
    String agentType  
    String payload  
}
```

Fluxo de Estado no Aplicativo macOS (SwiftUI)

Ilustrando como as `Views` do macOS conversam com o estado reativo.

```
flowchart LR  
    A((Eventos do <br/> Usuário)) --> B(ContentView)  
    B --> C{AppState <br/> ObservableObject}  
    C -->|Atualiza| D[ConversationViewModel]  
    C -->|Atualiza| E[PipelineView]  
    D --> F[ConversationView]  
    F --> A  
    E --> A  
    C --> G[(CoreData / Storage)]
```

Como Executar

Pré-requisitos

- macOS 14+ (Sonoma ou superior)
- Xcode 16+ ou Swift 6 Toolchain
- Node.js (para dependências em `src/` e `chat-cli`)

Instalação

1. Clone o repositório:

```
git clone <repo-url>  
cd mentalhealthos
```

2. Instale as dependências Node:

```
npm install
```

3. Para rodar a aplicação nativa (App):

```
swift run HealthOSApp  
# Ou abra o diretório no Xcode
```

4. Para rodar a CLI:

```
swift run Health0SCLI
```

Privacidade e Segurança

Este sistema foi arquitetado considerando as diretrizes de dados médicos:

- `SafetyReviewAgent` atua ativamente contra saídas perigosas ou anti-éticas.
- A persistência é regida por políticas explícitas (`memória bloqueada por padrão`).
- Modelos rodam através de runtimes locais/seguros com controle fino sobre a ingestão de contexto (`ContextArchitect`).